

| 全参数满血版DeepSeek-v3+vLLM企业部署

优势	描述
高级 GPU 优化	利用 CUDA 和 PyTorch 最大限度地提高 GPU 利用率，从而实现更快的推理速度。Ollama其实是对CPU-GPU的混合应用，但vllm是针对纯GPU的优化。
高级内存管理	通过PagedAttention算法实现对 KV cache的高效管理，减少内存浪费，从而优化大模型的运行效率。
批处理功能	支持连续批处理和异步处理，从而提高多个并发请求的吞吐量。
安全特性	内置 API 密钥支持和适当的请求验证，不像其他完全跳过身份验证的框架。
易用性	vLLM 与 HuggingFace 模型无缝集成，支持多种流行的大型语言模型，并兼容 OpenAI 的 API 服务器。

1. | 项目背景

甲方需求开发一个**ChatBI智能体**，需要模型有Text2SQL的能力，经测试，DeepSeek-V3版本的代码能力超越GPT-4o，故需要本地部署DeepSeek全参数版大模型。

2. | 部署方案

| 2.1 方案选择

采用DeepSeek-v3+vLLM方案

对大模型推理可以进行一个加速，效果比较好，比较稳定，支持相对比较高的并发。

| 2.2 资源评估

由于DeepSeek的框架原生采用 **FP8 训练**，因此仅提供 FP8 权重，预估仅700GB+显存便可轻松运行。当然也可以转换到BF16，在半精度下，需1400GB+显存，而量化到**int4**时需要450GB+。以下是半精度下显存占用情况：（占用 490G 显存，需要 7张 80G A100，租赁成本约1000元1天）



| 2.3 服务器准备

- 服务器硬件配置如下：
 - GPU：Nvidia A100（80G） GPU * 8
 - CPU：AMD EPYC 9K84 96-Core
 - 桥接方式：**NVLink（桥接）**
 - 内存：150G
 - 存储：2T
- 深度学习环境配置如下：
 - 操作系统：Ubuntu22.04

- PyTorch版本：2.5.1
- Python版本：3.12
- CUDA版本：12.4
- 其他软件包版本根据DeepSeek v3项目requirement决定。

(<https://github.com/deepseek-ai/DeepSeek-V3>)

- 多服务器部署方案 (**DeepSeek-V3+vLLM+RAY**)

如果需要部署的是fp8，或者其它的量化版本，一台服务器就不够了。这个时候需要，Ray+vLLM进行部署



地址：<https://github.com/ray-project/ray>



3. | 部署流程

| 3.1 下载权重文件

到魔搭社区下载：<https://www.modelscope.cn/>



| 注，下载前需提前留出600G左右存储空间，用于保存模型权重

在服务器上安装依赖，用于下载模型权重文件

```
pip install modelscope
```

执行命令，进行权重文件下载

```
mkdir ./deepseek
modelscope download --model OPEA/DeepSeek-V3-int4-sym-gptq-inc --local_dir
./deepseek
```

需要经过漫长的等待，才能下载完！

| 3.2 代码访问

```
import torch
from modelscope import AutoTokenizer, AutoModelForCausalLM,
GenerationConfig
model_name = "./deepseek"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name,
```

```

torch_dtype=torch.bfloat16, device_map="auto")
model.generation_config = GenerationConfig.from_pretrained(model_name)
model.generation_config.pad_token_id = model.generation_config.eos_token_id

messages = [
    {"role": "user", "content": "你好，请介绍下你自己！"}
]

input_tensor = tokenizer.apply_chat_template(messages,
add_generation_prompt=True, return_tensors="pt")
outputs = model.generate(input_tensor.to(model.device), max_new_tokens=100)
result = tokenizer.decode(outputs[0][input_tensor.shape[1]:],
skip_special_tokens=True)
print(result)

```

3.3 vLLM部署



地址: <https://github.com/vllm-project/vllm>

vLLM v0.6.6 支持在 NVIDIA 和 AMD GPU 上以 FP8 和 BF16 模式进行 DeepSeek-V3 推理。除了标准技术外，vLLM 还提供了管道并行性，允许你在多个网络连接的机器上运行该模型。

- vLLM安装

```
pip install vllm
```

目前vLLM已支持DeepSeek v3模型调用，可以在模型支持列表中查看模型关键字：

https://docs.vllm.ai/en/latest/models/supported_models.html



4. | 代码调用

| 直接使用vLLM推理（不方便）

```

from vllm import LLM

# For generative models (task=generate) only
llm = LLM(model=deepseek-ai/DeepSeek-V3, task="generate") # Name or path
of your model
output = llm.generate("Hello, my name is")
print(output)

```

| 提供OpenAI兼容的接口

创建兼容 OpenAI API 接口的服务器（接入到langchain、dify、扣子去开发）

```
python3 -m vllm.entrypoints.openai.api_server \
  --model deepseek-ai/deepseek-llm-671b \
  --host 0.0.0.0 \
  --port 8080 \
  --dtype auto \
  --tensor-parallel-size 2 \
  --trust-remote-code \
```

```
# 注意：请先安装 openai
# pip install openai
from openai import OpenAI

# 设置 OpenAI API 密钥和 API 基础地址
openai_api_key = "EMPTY" # 请替换为您的 API 密钥
openai_api_base = "http://localhost:8080/v1" # 本地服务地址

client = OpenAI(api_key=openai_api_key, base_url=openai_api_base)

models = client.models.list()
model = models.data[0].id
prompt = "描述一下北京的秋天"

# Completion API 调用
completion = client.completions.create(model=model, prompt=prompt)
res = completion.choices[0].text.strip()
print(f"Prompt: {prompt}\nResponse: {res}")
```

GPU显卡推荐：

